

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN **MÔN LẬP TRÌNH NÂNG CAO**

ĐỀ TÀI: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

Giảng viên lý thuyết: **Nguyễn Văn Sơn**
Giảng viên thực hành: **Phạm Bảo Phúc, Trần Ngọc Trúc Linh**

Mã lớp học phần: (2526II) UET.CS2043 3

Nhóm: 8

Lớp: K70I-IT3

Thành viên: Trần Anh Tú (nhóm trưởng)

Chu Quốc Hùng

Đinh Nho Quân

Bùi Duy Tuấn Anh

PHẦN 1: GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1.1. Mục tiêu thực hiện

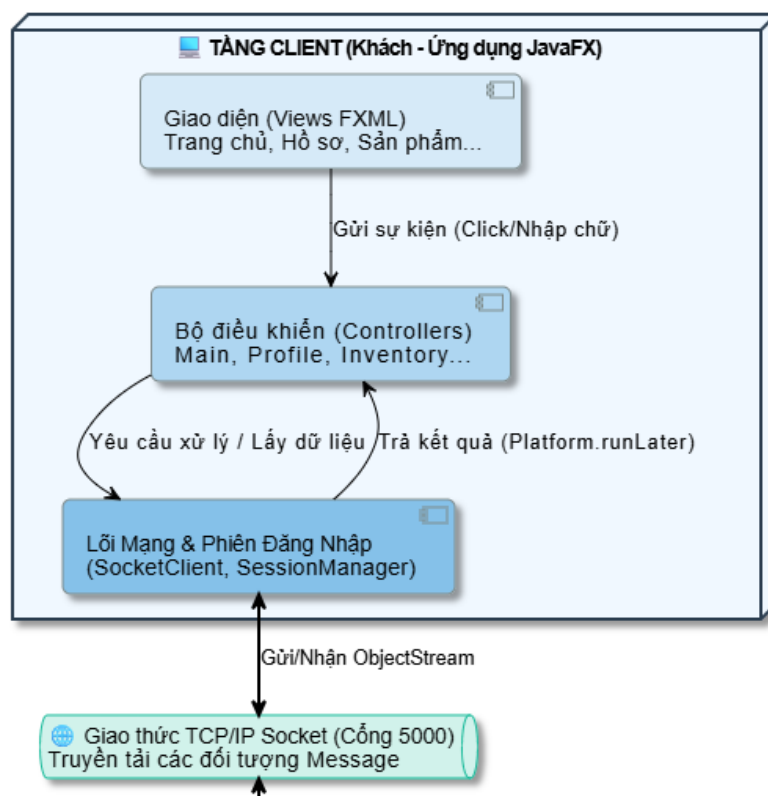
Dự án "Hệ thống Đấu giá Trực tuyến" được xây dựng nhằm giải quyết bài toán giao dịch tài sản trên không gian mạng một cách minh bạch, nhanh chóng và an toàn. Về mặt kỹ thuật, dự án đặt mục tiêu ứng dụng toàn diện các kiến thức nền tảng của kỹ thuật phần mềm, bao gồm: Lập trình hướng đối tượng (OOP), lập trình mạng (Socket TCP/IP), xử lý đa luồng (Multithreading), và áp dụng các mẫu thiết kế phần mềm (Design Patterns) để tạo ra một hệ thống Client-Server hoàn chỉnh.

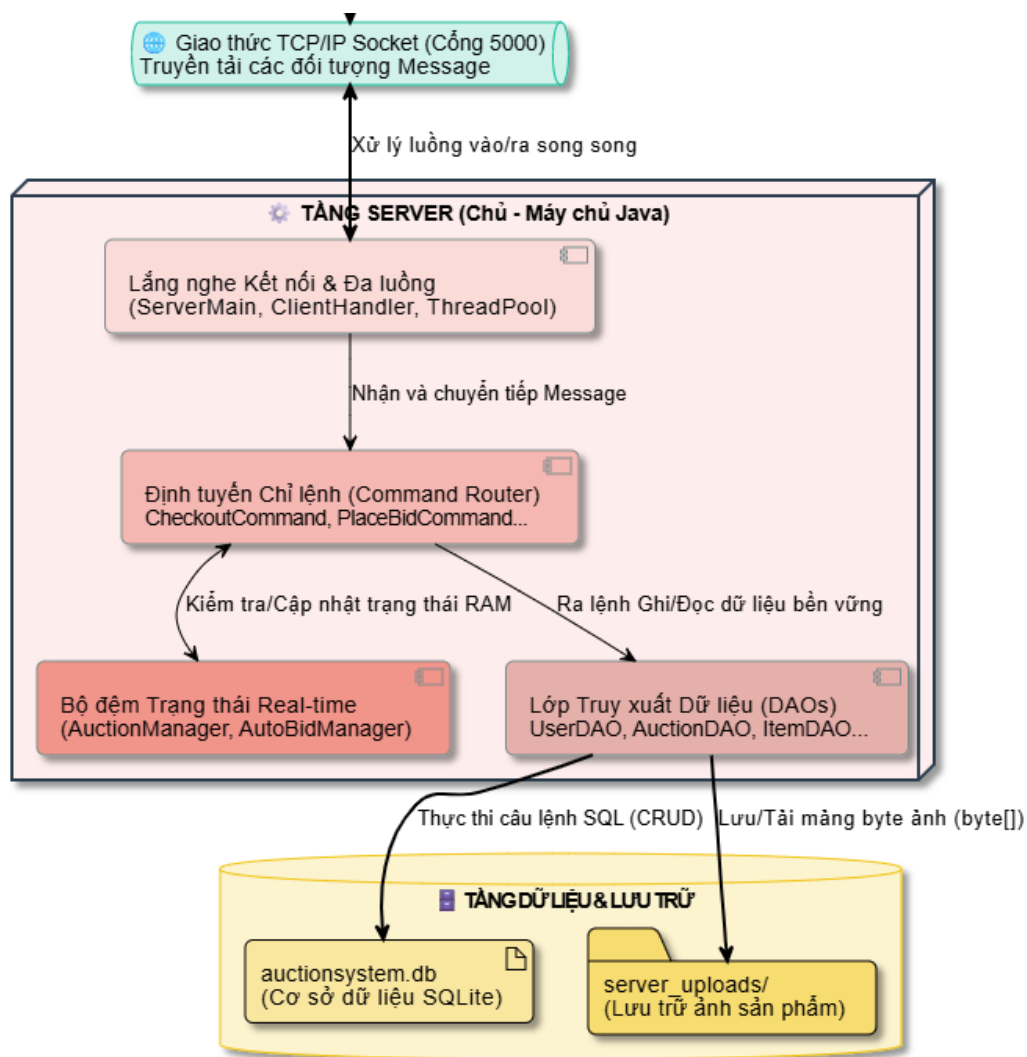
Hệ thống hướng tới việc xử lý dữ liệu theo thời gian thực (Real-time) với độ trễ thấp, đảm bảo tính nhất quán của dữ liệu tài chính và đem lại trải nghiệm mượt mà cho người dùng.

1.2. Phạm vi và Giới hạn thực hiện

- **Về nền tảng:** Ứng dụng Desktop được đóng gói và tối ưu cho Windows viết bằng ngôn ngữ Java, sử dụng JavaFX cho giao diện người dùng và SQLite cho hệ quản trị cơ sở dữ liệu nội bộ.
- **Về đối tượng người dùng:** Hệ thống phục vụ 3 nhóm định danh: Quản trị viên (Admin), Người bán (Seller), và Người mua (Bidder).
- **Về giới hạn phân loại vật phẩm:** Hệ thống trong phiên bản hiện tại được thiết kế giới hạn, chỉ tập trung hỗ trợ phân loại, quản lý và đấu giá cho 3 danh mục tài sản cụ thể bao gồm: **Nghệ thuật (Art)**, **Điện tử (Electronics)** và **Phương tiện (Vehicle)**. Các loại hình tài sản ngoài 3 danh mục này tạm thời không nằm trong phạm vi hỗ trợ của đồ án.
- **Về ranh giới chức năng:** Dự án tập trung mô phỏng luồng nghiệp vụ cốt lõi của một sàn đấu giá bao gồm: Đăng ký/đăng nhập, nạp/rút tiền giả lập qua ví điện tử, tạo phiên đấu giá, đặt giá thầu công và tự động (Auto-bid), thanh toán hóa đơn, và đẩy thông báo (Push Notification). Hệ thống không bao gồm các công thanh toán thực tế của ngân hàng hay các nghiệp vụ vận chuyển vật lý.

PHẦN 2: KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG





Mô tả chi tiết các thành phần trong kiến trúc

Phân hệ	Thành phần chính	Chức năng và Nhiệm vụ thực thi
1. TẦNG CLIENT (Khách - JavaFX)	Giao diện (Views FXML): Trang chủ, Hồ sơ, Sản phẩm...	Là lớp hiển thị trực quan, chịu trách nhiệm bố trí giao diện và trình bày dữ liệu cấu trúc cho người dùng cuối (Người bán, Người mua, Quản trị viên).
	Bộ điều khiển (Controllers): Main, Profile, Inventory...	Đóng vai trò điều phối, tiếp nhận các sự kiện tương tác (click, nhập liệu) từ giao diện, kiểm tra tính hợp lệ ban đầu và chuyển tiếp thành các yêu cầu logic.
	Lỗi Mạng & Phiên đăng nhập: SocketClient, SessionManager	Quản lý phiên làm việc hiện tại, duy trì trạng thái dữ liệu tạm thời trên RAM Client và trực tiếp gửi/nhận gói tin ngầm với Server bằng phương thức bất đồng bộ.
2. TẦNG MẠNG (Network Transport)	Giao thức TCP/IP Socket: Cổng 5000, Luồng ObjectOutputStream	Thiết lập luồng truyền tải dữ liệu liên tục và ổn định. Dữ liệu được đóng gói chuẩn hóa dưới dạng đối tượng Message giúp đồng bộ hai chiều theo thời gian thực.

Phân hệ	Thành phần chính	Chức năng và Nhiệm vụ thực thi
3. TẦNG SERVER (Chủ - Máy chủ Java)	Lắng nghe & Đa luồng: ServerMain, ThreadPool	Khởi tạo máy chủ, tiếp nhận kết nối mới và dùng cơ chế Thread Pool cấp phát cho mỗi Client một luồng riêng biệt chạy song song, chống nghẽn hệ thống.
	Định tuyến Chỉ lệnh: Command Router	Tiếp nhận gói tin, phân tích và định tuyến đến đúng lớp nghiệp vụ xử lý theo mẫu thiết kế Command Pattern (VD: CheckoutCommand, PlaceBidCommand).
	Bộ đệm Real-time: AuctionManager, AutoBidManager	Duy trì trạng thái các phiên đấu giá và điều phối robot tự động trả giá trực tiếp trên RAM máy chủ, đảm bảo tốc độ phản hồi tính bằng mili-giây.
	Truy xuất Dữ liệu (DAOs): UserDAO, AuctionDAO...	Tập hợp các lớp đối tượng giao tiếp với CSDL, nhận lệnh từ bộ định tuyến để chuẩn bị và thực thi các câu lệnh SQL đọc/ghi dữ liệu.
4. TẦNG DỮ LIỆU (Data & Storage)	Cơ sở dữ liệu (Database): Hệ quản trị SQLite (auctionsystem.db)	Lưu trữ bền vững các bảng dữ liệu có cấu trúc: thông tin tài khoản, ví tiền, lịch sử đặt giá, thông báo và thông tin vật phẩm.
	Kho lưu trữ tệp (Disk Storage): Thư mục vật lý (server_uploads)	Lưu trữ các tệp vật lý, đặc biệt là hình ảnh sản phẩm đấu giá dưới dạng mảng byte, phục vụ việc phân phối ảnh nhanh chóng cho Client.

PHẦN 3: ĐÁNH GIÁ CHI TIẾT THEO BAREM ĐIỂM MÔN HỌC

Nội dung đánh giá	Giải pháp Kỹ thuật Ứng dụng	Lý do lựa chọn hướng giải quyết & Ý nghĩa kiến trúc
Thiết kế lớp và cây kế thừa	Xây dựng cấu trúc phân cấp rõ ràng. Khởi tạo lớp cha User và các lớp con mở rộng Admin, Seller, Bidder. Khởi tạo lớp cha Item và các phân lớp chuyên biệt Art, Electronics, Vehicle.	Tối ưu hóa việc tái sử dụng mã nguồn (DRY). Dễ dàng mở rộng thêm các loại người dùng hoặc danh mục sản phẩm mới trong tương lai mà không cần sửa đổi mã nguồn cũ.
Áp dụng OOP (Encapsulation, Inheritance, Polymorphism, Abstraction)	- Encapsulation: Ẩn dữ liệu (private) và truy xuất qua getter/setter. - Inheritance: Tái sử dụng code qua extends. - Polymorphism: Ghi đè phương thức hiển thị đặc thù cho từng loại Item. - Abstraction: Sử dụng interface Command để trừu tượng hóa lệnh xử lý.	Tuân thủ tuyệt đối 4 tính chất cốt lõi của OOP. Giúp mã nguồn có tính Đóng gói cao (bảo vệ toàn vẹn dữ liệu), Dễ bảo trì và Mở rộng linh hoạt mà không làm phá vỡ logic nền tảng khi làm việc nhóm.

Nội dung đánh giá	Giải pháp Kỹ thuật Ứng dụng	Lý do lựa chọn hướng giải quyết & Ý nghĩa kiến trúc
Design Patterns phù hợp	- Singleton: Áp dụng cho SessionManager, AuctionManager. - Command Pattern: Phân rã luồng xử lý trên Server (PlaceBidCommand, CheckoutCommand...). - Factory Method: Khởi tạo lớp trừu tượng ItemFactory và giao việc tạo đối tượng cụ thể cho các lớp con (ArtFactory, ElectronicsFactory, VehicleFactory).	Mẫu Singleton tránh việc khởi tạo dư thừa tài nguyên và cung cấp điểm truy cập toàn cục. Command Pattern giúp loại bỏ cấu trúc switch-case khổng lồ, tách biệt đối tượng gọi lệnh và đối tượng thực thi. Factory Method giúp tuân thủ nguyên lý Open/Closed Principle), sau này nếu muốn bán thêm mặt hàng mới chỉ cần tạo thêm mà không cần sửa đổi mã nguồn cũ.
Quản lý người dùng, sản phẩm	Triển khai các thao tác CRUD cơ bản qua UserDAO và ItemDAO. Cung cấp chức năng đăng nhập, đăng ký, thay đổi mật khẩu, nạp tiền ví, đăng bán sản phẩm và phân loại kho đồ.	Tách biệt hoàn toàn logic nghiệp vụ (Business Logic) và logic truy xuất cơ sở dữ liệu (DAO). Dễ dàng thay đổi loại CSDL (từ SQLite sang MySQL) sau này nếu cần.
Chức năng đấu giá	Xây dựng vòng đời đầy đủ của phiên đấu giá: Khởi tạo mức giá gốc, định mức bước giá (step price), tiếp nhận lệnh Bid hợp lệ, cập nhật trạng thái kết thúc và luân chuyển tài chính (thanh toán).	Đáp ứng chính xác bài toán nghiệp vụ cốt lõi của một sàn giao dịch trực tuyến, kết nối cung-cầu với các ràng buộc về mặt thời gian và tài chính một cách minh bạch.
Xử lý lỗi & ngoại lệ	Xây dựng các Custom Exceptions tự định nghĩa (Ví dụ: InvalidBidException, InsufficientBalanceException). Sử dụng try-catch bao bọc kỹ các luồng truy xuất I/O mạng và CSDL.	Đảm bảo luồng mã nguồn (Clean Code) không bị phá vỡ. Việc ném lỗi tùy chỉnh giúp tách biệt quy trình xử lý lỗi trên Server và việc hiển thị Alert cảnh báo phù hợp cho người dùng trên Client.
Xử lý đấu giá đồng thời (concurrency)	Sử dụng khóa loại trừ tương hỗ synchronized(auction) tại luồng xử lý nhận lệnh Bid. Dùng kỹ thuật lồng khóa qua String.intern() ID người dùng khi thực thi giao dịch thanh toán.	Khóa đồng bộ là cơ chế sống còn giúp hệ thống loại bỏ hoàn toàn lỗi Race Condition (nhiều người đặt giá cùng một mili-giây gây đê dữ liệu) và chống Deadlock (tắc nghẽn chết chóc) trong luồng luân chuyển tiền tệ.
Realtime update (Observer/Socket)	Giao tiếp liên tục qua luồng mạng TCP/IP. Tại Client, áp dụng mẫu Observer Pattern : Các Controller đăng ký AuctionUpdateListener để "nghe lên" luồng mạng và sử dụng Platform.runLater() cập nhật lại giao diện tức thì.	Thay vì Client phải spam gửi request liên tục (Polling) gây nghẽn mạng, cơ chế "Đẩy" (Push) từ Server giúp tiết kiệm băng thông mạng và giao diện này số ngay lập tức.
Kiến trúc Client-Server	Tách biệt hoàn toàn dự án thành 2 module độc lập: Client Application và Server Application. Dữ liệu luân chuyển qua mạng	Kiến trúc mạng phân tán giúp nâng cao tính bảo mật (máy khách không nắm thông tin kết nối CSDL trực tiếp). Thread Pool giúp bảo vệ Server khỏi lỗi

Nội dung đánh giá	Giải pháp Kỹ thuật Ứng dụng	Lý do lựa chọn hướng giải quyết & Ý nghĩa kiến trúc
	được tuần tự hóa thành chuỗi Object. Cấp phát Fixed Thread Pool tại Server.	cạn kiệt bộ nhớ (OOM) khi số lượng người truy cập tăng đột biến.
MVC (JavaFX + FXML, Controller-Model-DAO)	Tổ chức cây thư mục chuẩn xác: Views là file đồ họa .fxml tách biệt; Controllers bắt sự kiện nút bấm; Models định nghĩa đối tượng; DAO xử lý logic cơ sở dữ liệu.	Phân tách mối quan tâm (Separation of Concerns). Việc thiết kế/sửa đổi đồ họa UI độc lập hoàn toàn với việc viết thuật toán lõi, giúp làm việc nhóm song song hiệu quả.
Maven/Gradle , coding convention	Dự án sử dụng Maven với tệp pom.xml để nạp tự động các thư viện (SLF4J, SQLite-JDBC, JavaFX). Áp dụng quy tắc đặt tên (CamelCase, PascalCase) chuẩn kỹ nghệ phần mềm.	Chuẩn hóa quy trình biên dịch và đóng gói phần mềm. Mã nguồn dễ đọc, đồng nhất, thuận tiện cho việc làm việc nhóm hoặc bàn giao đồ án.
Unit Test (JUnit)	Thiết lập các kịch bản kiểm thử (Test cases) bằng framework JUnit , giả lập môi trường đầu giá để kiểm chứng các hàm nghiệp vụ tài chính, đặt giá và khấu trừ số dư.	Giúp đo lường độ phủ của mã nguồn (Test Coverage), phát hiện sớm các lỗi thoái lui (Regression bugs) mỗi khi hệ thống có bản vá cập nhật tính năng mới.
CI/CD (GitHub Actions)	Tạo quy trình làm việc (Workflow) định dạng .yaml trên GitHub. Tự động kích hoạt chu trình mvn clean test và mvn package mỗi khi có mã nguồn mới được đẩy lên (Push/Pull Request).	Tự động hóa tích hợp và phân phối (DevOps). Đảm bảo nhánh chính (Main) không bao giờ bị lỗi build, rèn luyện tác phong phát triển phần mềm chuẩn doanh nghiệp.
Auto-Bidding	Tích hợp robot chạy ngầm sử dụng ScheduledExecutorService với cơ chế khóa tiền trích lập dự phòng. Ứng dụng kỹ thuật chống dội lệnh (Debouncing) để hủy luồng cược cũ nếu giá thay đổi liên tục.	Tiêu chí mở rộng sáng tạo. Giải phóng người dùng khỏi việc canh màn hình liên tục. Hỗ trợ hệ thống không bị tạo ra các "luồng mớ côi" làm treo ứng dụng.
Anti-sniping	Thiết lập logic kiểm đếm trên Server: Tự động cộng thêm lượng thời gian bù giờ (VD: 1 phút) nếu có một lệnh Bid mới xuất hiện ở 30 giây cuối cùng trước khi đóng phiên.	Chống lại các công cụ và hành vi gian lận cướp phiên ở tích tắc cuối. Đảm bảo tính công bằng tuyệt đối cho những người tham gia đấu giá thủ công. Tối đa hóa biên độ lợi nhuận công bằng cho người bán.
Bid History Visualization	Trích xuất dữ liệu lịch sử giá thầu, nạp vào thành phần LineChart hoặc BarChart của thư viện đồ họa JavaFX để vẽ biểu đồ đường xu hướng giá tăng dần theo thời gian thực.	Trực quan hóa dữ liệu thô (Data Visualization). Nâng tầm trải nghiệm người dùng (UX/UI), giúp họ nhanh chóng nắm bắt được độ nóng và xu hướng của phiên đấu giá.

PHẦN 4: PHÂN CÔNG CÔNG VIỆC GIỮA CÁC THÀNH VIÊN

BẢNG PHÂN CÔNG CÔNG VIỆC

Khối công việc	Công việc	Người làm (chính)
Hạ tầng lõi Hệ thống và Kết nối mạng	Xây dựng lõi Server đa luồng phục vụ kết nối Socket TCP/IP và định nghĩa cấu trúc gói tin mã hóa dữ liệu dùng chung giữa Client - Server.	Trần Anh Tú
	Thiết kế CSDL SQLite và viết các lớp truy xuất dữ liệu (DAO).	Trần Anh Tú
	Xử lý ghi log hệ thống.	Đinh Nho Quân
Xác thực và Quản lý tài khoản	Thiết kế và xử lý màn hình đăng nhập/đăng ký, xử lý logic kiểm tra ràng buộc dữ liệu đầu vào và xác thực quyền hạn từ Server.	Bùi Duy Tuấn Anh Đinh Nho Quân
	Thiết kế giao diện thông tin cá nhân và chức năng cập nhật mật khẩu.	Bùi Duy Tuấn Anh Chu Quốc Hùng
	Đồng bộ số dư ví tiền hiển thị lên thanh điều hướng chung.	Chu Quốc Hùng
Sàn đấu giá trực tiếp và Đặt giá thời gian thức	Thiết kế và xử lý điều hướng màn hình sảnh chính và các thẻ boc sản phẩm.	Bùi Duy Tuấn Anh
	Xử lý chức năng tìm kiếm và bộ lọc danh mục.	Đinh Nho Quân
	Thiết kế giao diện phòng đấu giá và xử lý các chức năng đấu giá cơ bản.	Bùi Duy Tuấn Anh
	Xử lý cơ chế cập nhật giá real-time liên tục qua khi có người đặt giá.	Trần Anh Tú
	Triển khai tính năng Đấu giá tự động (Auto-bid).	Đinh Nho Quân
	Triển khai thuật toán gia hạn giờ chốt chống cướp phiên (Anti-sniping)	Chu Quốc Hùng
	Tối ưu chức năng đặt giá và tích hợp đồ thị trực quan hóa diễn biến giá cuộc.	Trần Anh Tú
Kênh người bán và Quản lý kho hàng	Thiết kế và xử lý giao diện Kênh người bán và form thêm sản phẩm.	Bùi Duy Tuấn Anh
	Xử lý luồng truyền tải và lưu trữ hình ảnh sản phẩm lên máy chủ.	Đinh Nho Quân
	Thiết kế màn hình kho hàng cá nhân để theo dõi trạng thái các sản phẩm đang chạy hoặc đã đóng phiên.	Bùi Duy Tuấn Anh
	Xây dựng giao diện mô hình hóa dòng tiền và doanh thu thu về bằng biểu đồ trực quan.	Đinh Nho Quân
Giao dịch, Thông báo đẩy và Quản trị Admin	Triển khai cơ chế giam tiền tự động khi đặt giá và tra cứu lịch sử đặt giá.	Đinh Nho Quân Chu Quốc Hùng
	Xử lý khóa luồng trừ/cộng tiền an toàn khi chốt phiên, xử lý đơn hàng cần thanh toán.	Chu Quốc Hùng
	Thiết kế và xử lý chuông thông báo hỗ trợ real-time và offline.	Đinh Nho Quân
	Thiết kế và xử lý bảng điều khiển Admin cùng các chức năng liên quan.	Trần Anh Tú
Tối ưu tải hệ thống và Vận hành CI/CD	Tối ưu hóa tốc độ và bộ nhớ phần mềm.	Trần Anh Tú Đinh Nho Quân
	Thiết lập CI/CD qua GitHub Actions và cấu hình Checkstyle Google.	Đinh Nho Quân
	Viết các bài test kiểm thử phần mềm và đảm bảo độ bao phủ 60% (JaCoCo).	Chu Quốc Hùng